ONE LANGUAGE

ENTREGA FORMATIVA | PROPUESTA TECNICA

Propuesta tecnica

Arquitectura para conectar clientes web y movil, API, base de datos y servicio de reconocimiento de senas.

Campo	Detalle
Proyecto	One Language
Version	0.1 | borrador para revision del equipo
Fecha de corte tecnico	4 de noviembre de 2026
Alcance	Arquitectura, tecnologias, despliegue, seguridad, riesgos y evolucion.

Estado documental.
Basada en codigo local; requiere revision del equipo. No certifica pruebas ni disponibilidad. No contiene secretos o datos reales.

1. Resumen y arquitectura

Mantener una arquitectura modular: clientes para interaccion; backend para reglas y datos de cuenta; servicio IA para inferencia temporal; PostgreSQL para persistencia. El cliente coordina reconocimiento y guardado. La IA no lee ni modifica la base de datos.

Objetivos tecnicos

- Ofrecer experiencias web y Android/iOS con flujos funcionales equivalentes.
- Separar inferencia de vision por computador de la API transaccional.
- Proteger sesiones, grabaciones e historial por usuario.
- Desplegar componentes sin exponer PostgreSQL al navegador.
- Comunicar limites del vocabulario y del entorno de ejecucion.

Vista logica

Capa	Componentes	Canal
Presentacion	React/Vite web; React Native/Expo movil	HTTPS a onelanguage.tech; WSS a ai.onelanguage.tech
Entrada	Nginx y TLS en VPS	Rutas /auth, /api y /ws/recognize
Servicios	Spring Boot; Python/FastAPI	REST de negocio; WebSocket de reconocimiento
Datos	PostgreSQL; volumen para medios	Backend -> DB; backend -> almacenamiento

Principio. La prediccion es temporal. Solo se persiste cuando el usuario pide guardar; el backend autentica y liga el registro a la cuenta.

2. Tecnologias y justificacion

Tecnologia	Responsabilidad	Justificacion / tradeoff
React + Vite	Web SPA	Componentes y compilacion estatica rapida; requiere accesibilidad, rutas cliente y cuidado con storage web.
React Native + Expo	Android/iOS	Reutiliza ecosistema JS; Expo simplifica camara, voz y builds; plugins/permisos nativos pueden exigir rebuild.
Java 17 + Spring Boot	REST de negocio	Seguridad, validacion, JPA, correo y health; mayor huella que un servicio minimalista.
Python 3.11 + FastAPI	Inferencia y WebSocket	Integracion con ML y canal asincrono; TensorFlow/MediaPipe elevan tamano y uso de recursos.
TensorFlow/Keras + MediaPipe/OpenCV	Landmarks y clasificacion temporal	Usa rasgos de manos por frame; precisa datos representativos y calibracion por clase.
PostgreSQL 15	Persistencia relacional	Transacciones, integridad y relaciones; exige backups y migraciones controladas.
Liquibase	Versionado de esquema	Cambiosets trazables; cambios ya aplicados deben ser inmutables y respaldados.
Docker + Nginx + TLS	Empaquetado y entrega	Reproducibilidad, proxy y HTTPS; redes, secretos y rollback requieren coordinacion.

Decision: IA separada del backend

La API maneja identidad y datos durables; IA procesa frames y secuencias con dependencias de computo distintas. Separar permite aislar dependencias y escalar por separado, y evita que el modelo conozca credenciales de DB. El costo es operar dos servicios y mantener versionado el contrato de prediccion.

> Deuda tecnica. requirements.txt de IA incluye dependencias heredadas como Flask, Celery y Redis. Confirmar uso antes de retirarlas; no se afirma que todo el archivo sea necesario para el servicio actual.

3. Flujos e interfaces

Traduccion en vivo

Paso	Componente y accion	Interfaz
1	Cliente solicita permiso de camara y abre canal de reconocimiento.	WSS /ws/recognize; Origin permitido
2	Autentica el WebSocket y espera confirmacion.	Mensaje auth con JWT; respuesta ready
3	Frames se envian en orden; IA extrae landmarks y analiza secuencia.	Frame binario -> prediccion JSON
4	Interfaz muestra etiquetas/estado y opcion de voz.	Prediccion temporal, no persistida
5	Usuario decide guardar; cliente envia texto y grabacion opcional.	REST con Bearer JWT
6	Backend valida, persiste y devuelve; historial lee recursos propios.	POST/GET /api/translations

Contratos principales

Area	Rutas / contrato	Responsabilidad
Sesion	POST /auth/login; POST /api/users	Autenticacion y creacion; backend valida/firma JWT.
Perfil	GET /api/users/me	Identidad de sesion actual; no confiar en user id del cliente.
Historial	GET/POST/DELETE /api/translations	CRUD account-scoped y borrado logico.
Medios	POST /with-recording; GET /{id}/recording	Almacenar/recuperar grabacion con control de propietario.
IA	/health; /model-info; /ws/recognize	Salud, metadata y reconocimiento en vivo.

Flujo de datos. Los frames pertenecen al canal temporal. Al historial solo se envian datos que el usuario decide guardar; no se afirma que toda prediccion quede almacenada.

4. Despliegue y operacion

El modelo de produccion observado es una VPS con contenedores separados y Nginx del host como punto de entrada HTTPS. La web publica localmente; backend y DB permanecen detras del proxy/en redes Docker; IA utiliza un subdominio y canal WSS.

Entrada / servicio	Destino	Control operativo
https://onelanguage.tech	Nginx host -> web estatico	TLS, CSP, headers y rutas SPA.
/api/* y /auth/*	Proxy -> Spring Boot	REST, JWT, validaciones y logs.
wss://ai.onelanguage.tech/ws/recognize	Nginx -> Python:8000	Origin, JWT, limites y healthcheck.
Persistencia	Backend -> PostgreSQL interna	Volumen, entorno y migraciones Liquibase.
Medios	Backend -> volumen de grabaciones	Retencion y restauracion por formalizar.

Empaquetado observado

- Web: build Node 22 y runtime Nginx Alpine; imagen estatica; Compose usa red externa Docker para resolver backend.
- Backend: build Maven/Java 17 multietapa; datasource, JWT, correo y ruta media vienen del entorno.
- IA: Python 3.11; el modelo se monta solo lectura; healthcheck /health. Dockerfile local no declara usuario non-root.
- DB: PostgreSQL 15 con volumen y Liquibase para esquema. Mantener la base inaccesible desde Internet.
- La entrega coordina varios Compose/proyectos; no hay evidencia de un unico comando que restaure todos los servicios.

Despliegue y rollback recomendados

Respaldar DB y medios; fijar tags/commits; validar secretos fuera de Git; construir imagenes; desplegar un servicio por vez; probar salud y el flujo login -> traducir -> guardar -> historial; conservar la imagen anterior. El runbook debe incluir orden, responsable, verificacion y rollback.

5. Seguridad, calidad y riesgos

Atributo	Medida observada	Riesgo / trabajo pendiente
Autenticacion	JWT para REST; IA valida firma, expiracion y sujeto.	Web usa localStorage y movil AsyncStorage; evaluar cookie HttpOnly y SecureStore.
WebSocket	Origin allowlist, auth en timeout, limites por IP y frames/seg.	Proxy debe preservar upgrade e IP confiable; no confiar en headers arbitrarios.
Propiedad de datos	Consultas/borrados filtran por usuario autenticado.	Mantener pruebas negativas para historial y grabaciones entre cuentas.
Transporte/web	TLS/WSS, CSP, nosniff, X-Frame-Options y Permissions-Policy.	Revisar CSP/permisos cada release; minima exposicion de camara/microfono.
Secretos	Configuracion por variables de entorno.	Rotacion y purga Git coordinadas con responsables; no incluida aqui.
Continuidad	Volumenes persistentes en DB y medios.	Definir RPO/RTO, backups, cifrado, restore test y retencion de videos.
Capacidad IA	Secuencia landmarks; maximo 25 frames/s y limite concurrencia.	Medir latencia y CPU/RAM antes de ampliar usuarios.

Limitaciones observadas

- La metadata actual del modelo presenta siete etiquetas: Bien, Como estas, Gracias, Hola, Mal, Tu y Yo. No equivale a traduccion general de LSC.
- La IA depende de permiso de camara, red estable, modelo compatible y recursos CPU/RAM; healthcheck no mide precision linguistica.
- La operacion actual requiere coordinar servicios; healthchecks no reemplazan monitoreo ni rollback automatico.

Riesgo prioritario. Credenciales y tokens requieren gestion coordinada de cada servicio. Esta propuesta no contiene ni rota secretos y no declara que el historial Git este purgado.

6. Evolucion y criterios de aceptacion

Fase	Trabajo recomendado	Evidencia de cierre
1	Endurecer sesiones/secretos; storage seguro y perfiles prod.	Pruebas auth; rotacion por responsables; storage objetivo aprobado.
2	Unificar runbook, healthchecks, non-root, limites y rollback.	Deploy desde cero en QA; backup/restore y rollback ensayados.
3	Automatizar contratos REST/WS y rendimiento.	CI con auth, propietario, WS sin Origin/JWT, smoke y cobertura acordada.
4	Depurar dependencias; documentar modelo/dataset y ampliar vocabulario.	Inventario aprobado; metricas por clase, condiciones y sesgos.
5	Monitoreo, backups, retencion y privacidad de medios.	Alertas, politica, restore y borrado de medio verificados.

Criterios para aceptar la solucion

- Sin camara o token valido, el cliente muestra un estado accionable y no afirma exito.
- IA exige origen configurado y JWT vigente, responde contrato definido y limita abuso.
- Un usuario no puede leer, descargar ni eliminar traducciones/grabaciones de otra cuenta.
- Datos sobreviven recreacion de contenedores y un backup se restaura en QA.
- Builds web/movil usan URLs de entorno sin incluir secretos en bundle o repositorio.

Base tecnica revisada

Repositorio / rama	Revision	Fuentes
OneLanguage_Frontend / dev	b293cce	Dockerfile, nginx.conf, Compose, clientes web/movil
OneLanguage-Backend / dev	6175786	Dockerfile, Spring Boot, auth y traducciones
OneLanguage-Database / dev	196c9c1	PostgreSQL Compose, Liquibase y SQL
OneLanguage-AI / develop	781194b	Dockerfile, WebSocket, recognizer y pipeline

Pendiente: verificar que las revisiones citadas siguen vigentes y obtener aprobacion tecnica. Estos criterios son objetivos propuestos, no resultados de pruebas ya ejecutadas.